

INTERNSHIP REPORT

ON

Implementation of an Automated Testing

Yeshwantrao Chavan College of Engineering

Bachelor of Technology in Computer Technology



Submitted by:

Nayan Bhute

Supervised by:

Prof. Roshan S.

Bhanuse

(Faculty Supervised)

Sweta Thoppil

(Industry Supervisor)

Nagar Yuwak Shikshan Sanstha's

YESHWANTRAO CHAVAN COLLEGE OF ENGINEERING,

(An Automation Institution Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur)

NAGPUR – 441 110

CERTIFICATE OF APPROVAL

Certified that the Internship project entitled “**Automation of Coursera Web Application to Identify Courses**” has been successfully completed by **Nayan Bhute** during session 2025-26.

Faculty Supervisor

Signature :

Name : Prof. Roshan S. Bhanuse

Designation : Assistant Professor

Industry Supervisor

Signature : 

Name : Sweta Thoppil

Designation : Contractor, CPO L&D GenC Training

ACKNOWLEDGEMENT

I, **Nayan Bhute** would like to convey my gratitude to Yeshwantrao Chavan College of Engineering **Sweta Thoppil, Contractor CPO L&D GenC Training, Cognizant Technology Solution** for emphasizing on the Semester Internship Program and giving me the platform to interact with industry professionals.

I would also like to thank **Prof. Roshan S. Bhanuse** and **Dr. Kavita Singh** for giving me the opportunity to work on the prestigious Internship

I extend my warm gratitude and regards to everyone who helped me during my internship.

TABLE OF CONTENTS

Sr. No	Title	Page No
1	Title Page	1
2	Certificate of Approval	2
3	Acknowledgement	3
4	Internship Offer Letter	4 - 6
4	Index	7
5	Executive Summary	8-19
	• Introduction of the Report	8
	• Overview of the Internship providing Organization	12
	• Training Component	15
	• About the project	18
	• Weekly Work Log	25
	• Challenges and Solutions	27
	• Conclusion	29

INTRODUCTION OF THE REPORT

1.1 Background

The rapid growth of digital technology has transformed the way education and professional training are delivered across the globe. Online learning platforms such as Coursera have become integral to skill development, offering thousands of courses across diverse domains including programming, data science, language learning, and business management. As these platforms grow in scale and complexity, ensuring their functionality, reliability, and user experience becomes increasingly important.

In the software industry, Quality Assurance (QA) plays a vital role in ensuring that web applications perform as expected under various conditions. Manual testing, while essential, is time-consuming, repetitive, and prone to human error. Web automation testing addresses these limitations by allowing testers to simulate real user interactions programmatically, execute test scenarios consistently, and extract data efficiently from live web applications.

This internship was undertaken at Cognizant Technology Solutions, where the student was exposed to real-world automation testing practices. The project focused on automating key user workflows on the Coursera platform using browser automation tools, thereby gaining practical experience in the field of software testing and web automation.

1.2 Purpose of the Report

This report has been prepared to fulfil the requirements of the Semester Internship Program as prescribed by Yeshwantrao Chavan College of Engineering, Nagpur, under the Bachelor of Technology in Computer Technology program for the academic year 2025-26.

The report serves the following specific purposes:

- To provide a comprehensive and structured account of the internship experience undertaken at Cognizant Technology Solutions under the guidance of the industry and faculty supervisors.

- To document the problem statement, objectives, methodology, tasks performed, observations, and outcomes of the automation project carried out on the Coursera web application.
- To demonstrate the practical application of technical knowledge and skills acquired during the B.Tech program in a professional industry environment.
- To reflect upon the learning outcomes, challenges faced, and competencies developed during the internship period.

1.3 Objective of the Internship

The following objectives were defined at the commencement of the internship:

- To understand and apply web automation testing concepts in a real-world industry setting under professional supervision.
- To design and implement automation scripts for identifying and extracting course-related data from the Coursera platform.
- To handle dynamic web elements including search bars, dropdown menus, multi-window navigation, and web forms using browser automation techniques.
- To perform form validation testing by submitting incorrect input data and programmatically capturing the resulting error messages.
- To develop proficiency in industry-standard automation tools and frameworks, and to apply them effectively to solve practical QA problems.
- To enhance professional skills including documentation, problem-solving, analytical thinking, and collaboration within a team environment.

1.4 Scope of Work

The internship project was scoped around three specific automation tasks to be performed on the Coursera website (coursera.org):

Task 1 – Web Development Course Identification: To search for beginner-level web development courses offered in the English language on Coursera, and to extract and display

the course name, total learning hours, and rating for the first two courses appearing in the search results.

Task 2 – Language Learning Data Extraction: To navigate to the Language Learning category on Coursera, extract all available languages and their corresponding proficiency levels, and display the total number of courses available under each language and level combination.

Task 3 – Form Validation and Error Capture: To navigate from the Coursera homepage to the "For Enterprise" section, access the "Courses for Campus" product page under the Products menu, fill the "Ready to Transform" contact form with one intentionally invalid input (such as an incorrectly formatted email address), and programmatically capture and display the error or warning message generated by the form validation.

1.5 Tools and Technologies Used

Tool/Technology	Purpose
Playwright WebSocket	Browser automation and web element interaction
BDD(Behaviour Driven Development)	Writing test scenarios in plain English using Gherkin syntax (Given/When/Then)
TestNG	Test execution framework for managing and running automated test cases
Allure	Generating detailed HTML test execution reports with logs and results
GitHub	Version control and source code management
TypeScript	Primary programming language for automation scripting
Browser Driver	WebDriver for Google Chrome browser
Coursera.org	Target web application for testing
VS Code	Development and script execution environment

1.6 Methodology

The internship project was executed following a structured and systematic approach consisting of the following phases:

Phase 1 – Requirement Analysis: The problem statement was carefully analyzed to identify the web elements, pages, and interactions involved in each of the three automation tasks.

Phase 2 – Environment Setup: The required software tools, browser drivers, and automation frameworks were installed and configured on the development system to prepare the working environment.

Phase 3 – Script Development: Automation scripts were written to perform each task as per the defined scope, incorporating logic for element identification, data extraction, form interaction, and error capture.

Phase 4 – Execution and Observation: The developed scripts were executed on the live Coursera website. Outputs were observed, verified, and recorded for each task.

Phase 5 – Documentation: All findings, observations, and outcomes were systematically documented and compiled into this internship report as per the format prescribed by Yeshwantrao Chavan College of Engineering.

OVERVIEW OF THE INTERNSHIP PROVIDING ORGANIZATION

2.1 About Cognizant Technology Solutions

Cognizant Technology Solutions is one of the world's leading professional services companies, helping clients transform their business, operating, and technology models for the digital era. Headquartered in Teaneck, New Jersey, USA, Cognizant operates across multiple continents and serves clients in industries including banking and financial services, healthcare, retail, manufacturing, and technology. With a global workforce of over 300,000 professionals, Cognizant combines a passion for client satisfaction, technology innovation, deep industry expertise, and a community-minded approach to drive digital transformation.

Cognizant is listed on the NASDAQ stock exchange under the ticker symbol CTSI and is consistently ranked among the most admired companies in the IT services industry. The company has earned recognition for its culture of continuous learning, its investment in talent development, and its commitment to delivering high-quality technology solutions to clients across the globe.

2.2 Quality Engineering and Assurance (QEA) – Service Line

The internship was undertaken under the Quality Engineering and Assurance (QEA) service line of Cognizant, which is a dedicated division focused on ensuring the quality, reliability, and performance of enterprise applications, processes, and systems.

Cognizant's QEA service line offers a comprehensive range of services including intelligent and automated quality assurance, modernization assurance, and experience assurance. These services are designed to accelerate business and technology changes, improve customer experiences, and ensure regulatory compliance across various industry domains.

QEA places significant emphasis on automation and Artificial Intelligence to enhance testing efficiency and deliver high-quality outcomes. By leveraging these advanced technologies, the

QEA division helps businesses achieve digital success and maintain a competitive advantage in the rapidly evolving digital landscape. The division has successfully partnered with major enterprises across healthcare, finance, and retail sectors to transform their software quality assurance processes and achieve measurable improvements in software reliability, compliance, and time-to-market.

2.3 Generation Cognizant (GenC) – Internship Program

The internship was part of Cognizant's Generation Cognizant (GenC) program, a structured and comprehensive learning initiative designed to engage young talent with a well-defined technical learning pathway. The GenC program offers interns and fresh graduates the opportunity to interact with Subject Matter Experts (SMEs), understand the corporate environment, and develop the skills required to function as professional software engineers in a real-world setting.

The GenC program is built on the principle of Learner Autonomy, where students take charge of their own learning journey using available tools, resources, and self-paced platforms. The emphasis is more on "learning by doing" than on traditional instructor-led instruction, empowering interns to develop problem-solving skills through hands-on practice and real-world project exposure.

The program followed a Flipped Classroom approach, combining self-paced learning sessions with expert-guided practice sessions. During self-learning hours, interns studied the learning objectives independently through the Udemy platform and the Tekstac coding environment. This was followed by practice sessions conducted by SMEs, who provided guidance, resolved doubts, and helped interns apply the concepts to real-world problem statements.

2.4 Program Structure and Learning Journey

The GenC program followed a structured 65-day learning journey divided into two stages:

Stage 1 – Frontend with SQL covered fundamental programming concepts including JavaScript, Object-Oriented Programming, Collections, Exception Handling, File Handling, and SQL. This stage laid the foundation required to write efficient and maintainable automation code.

Stage 2 – Functional Testing with Playwright Automation Concepts covered the full spectrum of web automation testing, beginning with Functional Testing fundamentals and progressing through Selenium WebDriver, Page Object Model (POM), Apache POI, TestNG, BDD Cucumber, DevOps tools (Git, Maven, Jenkin), API automation using Postman and REST Assured, and Generative AI concepts.

The program concluded with a Business Aligned Project, which was divided into a Mini Project (individual activity) and a Hackathon (group activity). These projects were evaluated by Technical Subject Matter Experts from the Business Unit through Interim and Final Evaluations conducted via video interviews.

2.5 Internship Environment and Supervision

During the internship, the student worked under the guidance of **Sweta Thoppil**, Contractor, CPO L&D GenC Training, Cognizant Technology Solutions, who served as the Industry Supervisor. Academic supervision was provided by **Prof. Roshan S. Bhanuse**, Assistant Professor, Department of Computer Technology, Yeshwantrao Chavan College of Engineering, Nagpur, who served as the Faculty Supervisor.

The internship provided daily structured learning, hands-on exercises, code reviews, and project deliverable submissions. Regular mentor interactions and SME-connect sessions ensured that the learning experience was aligned with industry standards and professional best practices.

TRAINING COMPONENT

3.1 Topics Covered

The training was structured across a 65-day learning journey divided into two stages as defined by the Cognizant GenC QEA program:

Stage 1 – Frontend with SQL (Day 1 to Day 21)

- Core JavaScript Fundamentals: Data types, variables, operators, control flow statements
- Object-Oriented Programming: Encapsulation, Inheritance, Polymorphism, Abstraction
- Classes and Objects, Constructors, Keywords (this, super, final, static)
- Collections Framework: Map, Set, PriorityQueue
- Exception Handling: try-catch, finally, throw, throws, custom exceptions
- JavaScript File Handling: FileReader, FileWriter, BufferedReader, BufferedWriter
- JavaScript 8 Features: Lambda expressions, Stream API, Optional class
- SQL: DDL, DML, Joins, Subqueries, Aggregate Functions, Stored Procedures

Stage 2 – Functional Testing with Playwright Automation (Day 22 to Day 65)

- Agile/Scrum Methodology: SDLC, STLC, V-Model, Sprint planning, Retrospectives
- Functional Testing: White Box/Black Box Testing, Smoke, Sanity, Regression Testing
- JIRA: Project creation, Backlog management, Bug logging, Test case management
- Playwright WebDriver: Architecture, locators (XPath, CSS, ID, Name, LinkText)
- Advanced WebDriver: Actions class, JavascriptExecutor, Window/Frame handling, Waits
- Page Object Model (POM): Design pattern, Page Factory, reusable components
- Apache POI: Reading/writing Excel for data-driven testing
- TestNG: Annotations, assertions, XML configuration, parallel execution
- BDD Cucumber: Gherkin syntax, Feature files, Step Definitions, Hooks, Runners
- Automation Framework: Hybrid framework design, Extent Reports, Log4j
- DevOps: Git, GitHub, Maven, Jenkins CI/CD pipeline, Selenium Grid, Docker basics
- API Automation: Postman, REST Assured, JSON/XML parsing, Authentication
- GenAI: Fundamentals of Generative AI, GitHub Copilot, Prompt Engineering

3.2 Technologies and Tools Used

Tool / Technology	Category	Purpose
JavaScript (Azul JDK)	Programming Language	Core language for automation scripting
Selenium WebDriver 4.15.0	Automation Framework	Browser automation and element interaction
BDD Cucumber	Test Design	Behaviour-driven development using Gherkin
TestNG 7.4.0	Test Execution	Test management, grouping, parallel execution
ExtentReports	Reporting	HTML test execution reports with screenshots
Apache POI 5.2.4	Data Management	Reading/writing Excel test data
Maven	Build Tool	Dependency management and build automation
GitHub	Version Control	Source code management and collaboration
Jenkins	CI/CD	Automated test scheduling and pipeline
JIRA	Project Management	Sprint management, defect tracking, test cases
Eclipse IDE 2023-12	Development Environment	Writing, running, and debugging automation code
Postman	API Testing	Manual and automated API testing
REST Assured	API Automation	Java-based REST API automation framework
ChromeDriver	Browser Driver	WebDriver for Google Chrome browser

3.3 Certifications

During the internship period, the following assessments and evaluations were completed as part of the Cognizant GenC program:

- Stage 1 Assessment – Core Java and SQL (Cleared with qualifying score)
- Functional Testing ICT – Integrated Capability Test on Software Testing concepts
- Selenium ICT – Integrated Capability Test on Selenium WebDriver automation
- Interim Project Evaluation + Technical Evaluation – Video interview with BU SME (Green status)
- Final Project Evaluation + Final Technical Evaluation – Video interview with BU SME (Excellent)
- GenAI Fundamentals – Fundamentals of Generative AI [101-Basics] course on C-Learn

3.4 Week-wise Assessment and Scores

Week	Assessment / Activity	Topics Covered	Status
Week 1-2	JavaScript Code Challenges (CC) - Group 1 & 2	OOP, Collections, Exception Handling	Cleared
Week 3	ICT – JavaScript Assessment	Core Java comprehensive	Cleared
Week 3	SQL Code Challenges (DDL, DML, Joins, Subqueries)	SQL Queries	Cleared
Week 3	Stage 1 Assessment	JavaScript + SQL combined	Cleared
Week 4	Functional Testing ICT	STLC, Test cases, JIRA	Cleared
Week 5-6	Playwright Code Challenges	Locators, XPath, Actions	Cleared
Week 6	Playwright with POM	POM, Data-driven testing	Cleared
Week 7	Data Driven Code Challenge + ICT	TestNG, XML, JSON	Cleared
Week 7	Interim Project + Technical Evaluation	All Stage 2 topics + Mini Project	Green (Pass)
Week 9	GenAI Knowledge Check Assessment	Generative AI Fundamentals	Cleared
Week 10	Final Project + Technical Evaluation	Complete curriculum + Hackathon	Green(Pass)

ABOUT THE PROJECT

4.1 Project Title

Automation of Coursera Web Application to Identify Courses

4.2 Project Overview

This project was developed as part of the Hackathon in the Cognizant GenC QEA internship program. It focused on building a robust **web automation testing framework using Playwright** for the Coursera platform.

The objective was to automate real-world user workflows including search functionality, data extraction, dropdown handling, multi-page navigation, and form validation.

The framework was built using **TypeScript, Playwright Test Runner** for execution, and followed the **Page Object Model (POM)** design pattern to ensure maintainability and scalability. Reporting was handled using Playwright's built-in HTML reports.

4.3 Problem Statement

The project addressed the following problem statement as defined in the Hackathon:

"Search and display all web development courses. The courses should be at the beginner level, offered in the English language. Display the first two courses with course name, total learning hours, and rating."

The broader scope of the Hackathon extended this into three specific automation tasks:

Task 1:

Search for beginner-level web development courses in English on Coursera and extract the course name, total learning hours, and rating for the first two results displayed.

Task 2:

Navigate to the Language Learning category on Coursera, extract all available languages along with their proficiency levels and total course count, and display the structured data.

Task 3:

From the Coursera homepage, navigate to “**For Enterprise**” → “**Courses for Campus**”, fill in the “**Ready to Transform**” contact form with an intentionally invalid input (such as an incorrectly formatted email address), capture the validation error message, and display it.

4.4 Key Automation Scope

The automation framework for this project was implemented using **Playwright**, leveraging its modern features to handle dynamic web applications efficiently. The key technical aspects covered are as follows:

Browser and Multi-Page Handling:

Handling multiple browser windows and tabs was a core requirement due to dynamic navigation on the Coursera platform. Playwright’s **browser context and multi-page handling capabilities** were used to seamlessly manage multiple tabs without complex switching logic, improving reliability compared to traditional approaches.

Auto-Waiting and Dynamic Content Handling:

Unlike traditional frameworks, Playwright provides **built-in auto-waiting**, which ensures elements are ready before interaction. This eliminated the need for explicit waits and improved test stability while working with dynamically loaded content such as search results and filters.

Dropdown Handling and Data Extraction:

In Task 2, multiple dropdown filters containing languages and proficiency levels were handled using Playwright’s **locator strategies and built-in methods**. Data was extracted efficiently and stored in structured formats such as arrays and objects (equivalent to collections like ArrayList and HashMap) for better reporting and validation.

Navigation and Page Control:

The framework implemented smooth navigation between pages using Playwright’s **page navigation APIs**, including page.goto() and browser history controls. This ensured realistic user journey simulation while maintaining execution speed.

Form Interaction and Validation:

Task 3 involved interacting with a complex multi-field web form. Playwright's **robust element locators and input handling methods** were used to fill text fields, select dropdown values, and submit the form. Invalid input was intentionally provided to trigger validation logic.

Validation Message Capture:

Playwright's **built-in assertions and text extraction methods** were used to capture and validate error messages displayed on the UI. This ensured accurate verification of form validation behavior.

Scrolling and Element Visibility:

Playwright automatically handles scrolling into view before interacting with elements. Additionally, **built-in scrolling support and locator actions** were used to interact with elements not initially visible in the viewport, eliminating the need for custom JavaScript execution in most cases.

Parallel Execution and Performance:

The framework leveraged Playwright's **parallel test execution and lightweight architecture**, resulting in faster execution compared to traditional tools. This improved overall efficiency during test runs.

4.5 Tools and Technologies Used

Tool/Technology	Purpose
Playwright	Browser automation and web element interaction
BDD(Behaviour Driven Development)	Writing test scenarios in plain English using Gherkin syntax (Given/When/Then)
WebSocket	Test execution framework for managing and running automated test cases
Allure	Generating detailed HTML test execution reports with logs and results
GitHub	Version control and source code management

TypeScript	Primary programming language for automation scripting
BrowserDriver	WebDriver for Google Chrome browser
Coursera.org	Target web application for testing
VS Code	Development and script execution environment
Apache POI 5.2.4	Reading and writing test data from Excel files for data-driven testing
JIRA	Agile project management, test case documentation, sprint management, and defect tracking
Maven	Build automation and dependency management
Page Object Model (POM)	Design pattern for separating test logic from page element definitions

4.6 Framework Architecture

The automation framework for this project was designed and implemented using **Playwright with TypeScript**, incorporating modern automation capabilities to handle dynamic web applications efficiently.

Handling multiple browser windows and search functionality was a core requirement, as the Coursera platform dynamically loads content and may open links in the same or new tabs. Playwright's **browser context and multi-page handling features** were used to manage multiple tabs seamlessly without complex switching logic.

Extracting multiple dropdown list items and storing them in collections was implemented in Task 2 using Playwright's **powerful locator API and locator().allTextContents() methods**. The extracted data was stored in structured formats such as arrays and objects for organized reporting.

Navigating back to the homepage between tasks was achieved using Playwright's **page.goto() and navigation APIs**, ensuring smooth and fast transitions without relying on browser history alone.

Form interaction in Task 3 was handled using Playwright's **robust element locators and input handling methods**, allowing interaction with text fields, dropdowns, and buttons across multiple page sections efficiently.

Capturing warning messages was achieved using Playwright's **built-in assertions (expect) and text extraction methods**, ensuring reliable validation of dynamic error messages displayed on the UI.

Scrolling through web pages was simplified using Playwright's **automatic scrolling and element interaction behavior**, which ensures elements are brought into view before performing actions, reducing the need for manual JavaScript execution.

4.7 Task 1 – Web Development Course Identification

Objective:

To search for beginner-level web development courses offered in English and extract the course name, total learning hours, and rating for the first two results.

Steps Performed:

The browser was launched using Playwright, and the Coursera homepage was opened using `page.goto()`. The search bar was located using Playwright's locator strategy, and the keyword "Web Development" was entered and submitted.

Filters such as Beginner level and English language were applied using text-based locators. Playwright's auto-waiting feature ensured that elements were interactable before performing actions.

The page dynamically loaded course results, and Playwright automatically waited for elements to be visible. The first two course cards were identified using locators, and the course name, learning hours, and rating were extracted using `textContent()` methods.

The extracted data was stored in arrays and printed to the console. Playwright's built-in reporting captured the execution results.

Challenges Faced:

The page used lazy loading for course cards. This was handled efficiently using Playwright's auto-waiting and `locator().nth()` methods, eliminating the need for explicit waits or manual scrolling logic.

4.8 Task 2 – Language Learning Data Extraction

Objective:

To extract all available languages, proficiency levels, and course counts from the Language Learning section.

Steps Performed:

The script navigated to the Language Learning section using Playwright's navigation methods. Language filters and dropdown elements were identified using **CSS and text-based locators**.

Playwright's **locator().allTextContents()** and **iteration methods** were used to extract all language names and their corresponding levels. Course counts were captured alongside each filter option.

The extracted data was stored in structured objects (similar to HashMap) and displayed in a formatted manner.

Challenges Faced:

The filter panel contained dynamically rendered elements. Playwright handled this using **auto-waiting and locator-based scrolling**, ensuring all elements were accessible without additional JavaScript execution.

4.9 Task 3 – Form Validation and Error Message Capture

Objective: To validate error handling by submitting invalid input in the contact form and capturing the displayed error message.

Steps Performed:

From the homepage, navigation to **“For Enterprise”** → **“Courses for Campus”** was performed using Playwright locators.

The page was scrolled automatically, and the **“Ready to Transform”** form was located. Form fields were filled using Playwright's **fill()** method, and an invalid email was entered intentionally.

After submitting the form, Playwright's **expect() assertion** was used to capture and validate the error message displayed dynamically.

The error message was extracted using **textContent()** and logged in the test output.

Challenges Faced:

The form and validation messages were dynamically rendered. Playwright's **auto-waiting and built-in assertions** ensured reliable detection and validation without requiring explicit waits.

4.10 Test Execution and Reporting

All automation tasks were executed using the **Playwright Test Runner**, which supports parallel execution and efficient test management.

Playwright's **built-in HTML reporter** generated detailed execution reports, including test results, logs, and screenshots. Screenshots and traces were automatically captured for failed tests using Playwright's **trace viewer and debugging tools**.

The project was integrated with **Jenkins** for continuous integration, enabling scheduled and on-demand test execution. The source code was maintained on **GitHub**, following standard version control practices.

4.11 Defect Management Using JIRA

All defects identified during execution were logged in **JIRA**, including details such as defect summary, steps to reproduce, expected vs actual results, severity, priority, and supporting evidence.

Defect tracking followed Agile methodology, with statuses managed across stages such as Open, In Progress, Resolved, and Closed.

WEEKLY WORK LOG

Week	Dates	Tasks Performed	Learning Outcomes	Supervisor Remarks
1	31 Jan - 4 Feb	Corporate induction, Icebreaker sessions, Core JavaScript - OOP, data types, variables, constructors	Understood OOP principles, Java basics and JVM architecture	Good progress
2	7 Feb - 11 Feb	Core JavaScript - Collections, Exception Handling, File Handling, JDBC, hands-on exercises on Tekstac	Implemented ArrayList, HashMap, try-catch blocks, file I/O operations	Satisfactory
3	14 Feb - 18 Feb	SQL basics, DDL/DML, Joins, Stage 1 Assessment	Mastered lambda expressions, stream API, and SQL query writing	Good
4	21 Feb - 25 Feb	Agile/Scrum, STLC, Functional Testing, JIRA project setup, test case writing	Understood testing lifecycle, created test cases and sprint in JIRA	Good
5	28 Feb - 4 Mar	Playwright WebDriver setup, locators (XPath, CSS, ID), browser automation basics	Wrote first Selenium scripts, identified elements using multiple locator strategies	Excellent
6	7 Mar - 11 Mar	POM design pattern, Page Factory, Apache POI, data-driven testing, advanced XPath	Built first POM framework, read Excel test data, implemented dynamic XPath locators	Excellent
7	14 Mar - 18 Mar	TestNG annotations, assertions, XML config, BDD Cucumber feature files, step definitions	Integrated TestNG with Selenium, wrote first BDD feature file with Gherkin syntax	Excellent
8	21 Mar - 25 Mar	Hackathon project - Task 1 development, Course search and data extraction automation	Automated Coursera course search with filters, extracted course data into ArrayList	Excellent
9	28 Mar - 1 Apr	Hackathon - Task 2 (Language extraction) and Task 3 (Form validation) development	Extracted language data using HashMap, captured form	Excellent

			validation error message	
10	4 Apr - 8 Apr	Git, Maven, Jenkins pipeline setup, Selenium Grid, ExtentReports integration	Pushed project to GitHub, configured Jenkins CI job, generated HTML ExtentReport	Excellent
11	11 Apr - 15 Apr	API automation with Postman and REST Assured, Docker basics, GenAI fundamentals	Wrote REST Assured test scripts, learned Docker containerization concepts and GenAI tools	Excellent
12	18 Apr - 22 Apr	Final project delivery, code review, documentation, Final Project and Technical Evaluation	Successfully delivered complete automation framework, cleared Final Evaluation with Excellent	Excellent

CHALLENGES AND SOLUTIONS

8.1 Challenge 1 – Dynamic Lazy Loading on Coursera

One of the major challenges encountered during the project was handling **lazy-loaded content** on the Coursera platform. The search results page does not load all course elements at once; instead, additional course cards are rendered dynamically as the user scrolls down the page. In traditional automation approaches, this often leads to synchronization issues where the script tries to access elements that are not yet present in the DOM, resulting in failures.

This challenge was effectively handled using Playwright's **auto-waiting mechanism and locator strategies**. Playwright automatically waits for elements to be attached to the DOM, visible, and stable before performing any interaction. Instead of using explicit waits, locators such as `locator().nth()` and `locator().first()` were used to target specific course elements. Additionally, Playwright ensures that elements are interactable before executing actions, which significantly reduces flakiness. This built-in synchronization eliminated the need for manual waiting logic and ensured reliable extraction of dynamically loaded course data.

8.2 Challenge 2 – Nested Dropdown Data Extraction

Another challenge was extracting structured data from **nested dropdown filters** in the Language Learning section. These dropdowns contained multiple levels of information, including languages, proficiency levels, and course counts. Some elements were dynamically rendered and not immediately visible, making data extraction complex.

This was addressed using Playwright's **locator chaining and iteration capabilities**. By chaining locators (e.g., locating a parent container and then targeting child elements within it), the script was able to accurately traverse the DOM hierarchy. Methods like `locator().allTextContents()` and looping constructs were used to iterate through all available

options. The extracted data was stored in structured formats such as JavaScript objects and arrays, preserving the hierarchical relationship between languages and their levels. Playwright's ability to handle dynamic DOM updates ensured that all dropdown elements were captured without missing any data.

8.3 Challenge 3 – Locating the Form Below the Page Fold

Many elements on the Coursera platform, such as course cards and form fields, are not immediately visible within the initial viewport. Attempting to interact with such elements in traditional frameworks often results in errors like “element not interactable.”

Playwright resolves this issue through its **automatic scrolling and actionability checks**. Before performing any action (such as click, fill, or hover), Playwright automatically scrolls the element into view and verifies that it is visible, enabled, and stable. This eliminates the need for manually writing scrolling logic using JavaScript. Additionally, Playwright's actionability checks ensure that interactions only occur when the element is fully ready, thereby improving test reliability and reducing execution errors.

8.4 Challenge 4 – Capturing the Validation Error Message

Capturing validation error messages in web forms was another key challenge. In Task 3, the error message for an invalid email input was dynamically generated and only appeared after form submission. Such elements are often not present in the DOM initially and may have unstable locators.

This challenge was handled using Playwright's **built-in assertion library and locator-based waiting**. After submitting the form, the script used `expect(locator).toBeVisible()` to wait for the error message to appear. Once visible, the text was extracted using `textContent()` and validated against expected values. Playwright's assertions automatically retry until the condition is met or a timeout occurs, ensuring reliable detection of dynamic elements without requiring explicit waits. This approach simplified validation logic and improved accuracy in capturing error messages.

8.5 Challenge 5 – Environment Setup Delays

Setting up the automation environment can often be time-consuming and error-prone, especially when dealing with browser driver compatibility issues in traditional tools.

Playwright simplifies this process significantly through **single-command installation approach**. By running `npx playwright install`, all required browser binaries (Chromium, Firefox, WebKit) are automatically downloaded and configured. Unlike traditional frameworks, there is no need to

manually manage browser drivers such as ChromeDriver. This eliminates version mismatch issues and ensures a consistent execution environment across different systems.

Additionally, Playwright provides a built-in test runner, reporting tools, and debugging capabilities (such as trace viewer), reducing the need for external dependencies. This streamlined setup process allowed faster project initialization and improved overall developer productivity.

CONCLUSION

This internship project provided comprehensive hands-on experience in designing and implementing a modern web automation framework using Playwright. By transitioning from traditional automation approaches to Playwright, the project significantly improved execution speed, test reliability, and overall efficiency. Playwright's advanced capabilities—such as auto-waiting, parallel execution, and built-in reporting—helped eliminate common challenges like synchronization issues, flaky tests, and complex setup processes, resulting in a more robust and maintainable automation solution.

Throughout the project, practical expertise was developed in handling real-world automation scenarios, including dynamic web element interaction, lazy-loaded content, multi-page and multi-tab navigation, structured data extraction, and complex form validation. The use of Playwright's powerful locator strategies and built-in assertions ensured accurate and stable test execution across different workflows on the Coursera platform. Additionally, the implementation of the Page Object Model (POM) design pattern improved code reusability, readability, and scalability, aligning the framework with industry-standard best practices.

Beyond technical implementation, the project also strengthened analytical thinking and problem-solving abilities. Various challenges—such as handling dynamic DOM updates, extracting nested dropdown data, and capturing runtime validation messages—were addressed using efficient and optimized approaches provided by Playwright. This experience enhanced the ability to debug issues, design reliable test strategies, and adapt quickly to modern automation tools and frameworks.

In conclusion, this internship served as a transformative learning experience that bridged the gap between academic knowledge and industry practices. It established a strong foundation in modern test automation using Playwright and equipped with the necessary skills to build scalable, maintainable, and high-performance automation frameworks. The knowledge and experience gained during this project will be highly valuable for future roles in software testing, quality engineering, and automation development.